

LAB2 실험 후 보고서

순차논리 8종 — 업/다운 카운터부터 7세그먼트 자리 스캔까지

이름
형종희

학번
2023440145

조
C조

실습일
2026-09-21

작성일
2026-09-27

대상 보드
Combo II-DLD S75 ·
xc7s75fpga484-1

GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
구조	LAB1/ 과 LAB2/ 가 형제 폴더로 구성됨
이 보고서 반영 커밋	dac086e — README, LAB2 회로 소스(RTL·TB·XDC), 보드 검증 영상 8개, 이 결과보고서 자체
LAB2 경로	LAB2/circuits/LAB2-01~08/ , LAB2/evidence/videos/ , LAB2/LAB2_실험후보고서.html
제출 태그	lab2-submit  (커밋 dac086e)

0. 8개 회로 결과 요약

회로	Behavioral Sim	구현	Setup WNS
01 업/다운 카운터	36개 · 356 ns	Complete	999997.00 ns
02 클록 분주	129개 · 436 ns	Complete	999996.31 ns
03 레지스터	7개 · 66 ns	Complete	999997.81 ns
04 시프트 레지스터	8개 · 106 ns	Complete	999997.00 ns
05 PISO	114개 · 976 ns	Complete	999997.50 ns
06 Moore 상태 머신	23개 · 226 ns	Complete	999997.25 ns
07 Mealy 상태 머신	11개 · 67 ns	Complete	999997.75 ns
08 7세그먼트 자리 스캔	194개 · 1306 ns	Complete	999998.06 ns

목차

- | | |
|------------------|---------------------|
| 1. 11 · 업/다운 카운터 | 5. 15 · PISO |
| 2. 12 · 클록 분주 | 6. 16 · Moore 상태 머신 |
| 3. 13 · 레지스터 | 7. 17 · Mealy 상태 머신 |
| 4. 14 · 시프트 레지스터 | 8. 18 · 7세그먼트 자리 스캔 |

1. 실험 11 · 업/다운 카운터

①개요

4비트 업/다운 카운터. rst가 최우선, enable일 때 down 여부에 따라 15↔0을 순환하며 증가·감소한다.

포트	핀/제어
I/O	clk, rst, enable, down → value[3:0]
보드 매핑	LED[3:0] = value, N8 = enable, K4 = rst, SW8 = down

②RTL

```
module counter4 (  
    input wire clk, rst, enable, down,  
    output reg [3:0] value  
);  
    always @(posedge clk) begin  
        if (rst) value <= 4'd0;  
        else if (enable) begin  
            if (down) value <= value - 4'd1;  
            else value <= value + 4'd1;  
        end  
    end  
endmodule
```

③Behavioral Simulation 결과



VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	36개
종료 시각	356 ns (= 356000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```
RUN C:\iverilog\bin\vvp.EXE -N C:\Users\UOS\Documents\lab2_01_counter\build\sim\run-fbadcc9e780f422baef39009fac806e6\sim.vvp  
VCD info: dumpfile wave.vcd opened for output.  
LAB2_PASS counter4 checks=36  
C:\Users\UOS\Documents\lab2_01_counter\sim\tb counter4.sv:19: $finish called at 356000 (1ps)
```

실제 실행 로그 — LAB2_PASS counter4 checks=36 , 356 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	11	48,000	0.02%
Bonded IOB	12	338	3.55%

⑤구현(Implementation)·DRC 결과

✔ write_bitstream Complete

항목	값
Setup WNS	999997.000 ns
Hold WHS	0.122 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✔ 비트스트림 생성 완료 — 3,687,015 bytes , 00:17:30 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED[3:0] = value, N8 = enable, K4 = rst, SW8 = down로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-01 업/다운 카운터 보드 동작 영상 (evidence/videos/LAB2-01_counter.mp4)

[GitHub에서 영상 열기 ↗](#)

2. 실험 12 · 클록 분주

①개요

DIVISOR로 클록을 나누어 duty 50%의 divided 출력과, 소비 가능한 한 클록 폭의 tick을 만든다. 보드에서는 ÷2·÷10·÷50·÷1000 네 개를 동시에 인스턴스화했다.

포트	핀/제어
I/O	clk, rst → divided, tick
보드 매핑	LED[3:0] = ÷2·÷10·÷50·÷1000, LED4 = tick (자동 동작, 버튼 불필요)

②RTL

```

module clock_divider #(parameter integer DIVISOR = 10) (
    input wire clk, rst,
    output reg divided,
    output wire tick
);
    localparam integer WIDTH = $clog2(DIVISOR);
    reg [WIDTH-1:0] count;
    assign tick = !rst && (count == DIVISOR - 1);
    always @(posedge clk) begin
        if (rst) begin count <= 0; divided <= 1'b0; end
        else begin
            if (count == DIVISOR - 1) count <= 0; else count <= count + 1'b1;
            if (count == DIVISOR/2 - 1) divided <= 1'b1;
            else if (count == DIVISOR - 1) divided <= 1'b0;
        end
    end
endmodule

```

③Behavioral Simulation 결과

 **PASS** VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	129개
종료 시각	436 ns (= 436000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```

RUN C:\iverilog\bin\vvp.EXE -N C:\Users\UOS\Documents\lab2_02_clock_divider\build\sim\run-161ff9a1e9ba4c3789cf0c28e7696202\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS clock_divider checks=129
C:\Users\UOS\Documents\lab2_02_clock_divider\sim\tb_clock_divider.sv:19: $finish called at 436000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정당 여부를 확인하세요.
WAVE C:\Users\UOS\Documents\lab2_02_clock_divider\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.

```

실제 실행 로그 — LAB2_PASS clock_divider checks=129 , 436 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	29	48,000	0.06%
Bonded IOB	10	338	2.96%

⑤구현(Implementation)·DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999996.312 ns
Hold WHS	0.122 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✓ 비트스트림 생성 완료 — 3,687,021 bytes , 00:24:26 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED[3:0] = ÷2·÷10·÷50·÷1000, LED4 = tick (자동 동작, 버튼 불필요)로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-02 클록 분주 보드 동작 영상 (evidence/videos/LAB2-02_clock_divider.mp4)

[GitHub에서 영상 열기 ↗](#)

3. 실험 13 · 레지스터

①개요

load로 stored에 저장, transfer로 stored를 value에 전달한다. 두 제어가 동시에일 때 value는 에지 직전의 stored를 받는다(입력이 아니라 저장값).

포트	핀/제어
I/O	clk, rst, load, transfer, data_in[3:0] → stored[3:0], value[3:0]
보드 매핑	LED[7:4] = stored, LED[3:0] = value, SW8 = load, SW7 = transfer, SW[7:4] = data_in

②RTL

```
module register_pair (  
    input wire clk, rst, load, transfer,  
    input wire [3:0] data_in,  
    output reg [3:0] stored, value  
);  
    always @(posedge clk) begin  
        if (rst) begin stored <= 4'd0; value <= 4'd0; end  
        else begin  
            if (load) stored <= data_in;  
            if (transfer) value <= stored;  
        end  
    end  
endmodule
```

③Behavioral Simulation 결과



VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	7개
종료 시각	66 ns (= 66000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```
RUN C:\iverilog\bin\vvp.EXE -N C:\Users\UOS\Documents\lab2_03_register\build\sim\run-d263807af8044a09bf958779bd95473d\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\UOS\Documents\lab2_03_register\sim\tb_register_pair.sv:19: $finish called at 66000 (1ps)
```

실제 실행 로그 — LAB2_PASS register_pair checks=7 , 66 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	11	48,000	0.02%
Bonded IOB	17	338	5.03%

⑤구현(Implementation)·DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999997.812 ns
Hold WHS	0.119 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✔ 비트스트림 생성 완료 — 3,687,016 bytes , 00:28:22 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED[7:4] = stored, LED[3:0] = value, SW8 = load, SW7 = transfer, SW[7:4] = data_in로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-03 레지스터 보드 동작 영상 (evidence/videos/LAB2-03_register.mp4)

[GitHub에서 영상 열기 ↗](#)

4. 실험 14 · 시프트 레지스터

①개요

enable일 때 매 클럭 {serial_in, value[3:1]}로 갱신해, 입력이 bit3으로 들어와 bit0 방향으로 한 칸 씩 이동한다.

포트	핀/제어
I/O	clk, rst, enable, serial_in → value[3:0]
보드 매핑	LED[3:0] = value, SW1 = serial_in, N8 = enable

②RTL

```

module shift_register4 (
    input wire clk, rst, enable, serial_in,
    output reg [3:0] value
);
    always @(posedge clk) begin
        if (rst) value <= 4'd0;
        else if (enable) value <= {serial_in, value[3:1]};
    end
endmodule

```

③ Behavioral Simulation 결과



VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	8개
종료 시각	106 ns (= 106000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```

LAB2_PASS shift_register4 checks=8
C:\Users\UOS\Documents\lab2_04_shift_register\sim\tb_shift_register4.sv:19: $finish called at 106000 (1ps)

```

실제 실행 로그 — LAB2_PASS shift_register4 checks=8 , 106 ns 종료.

④ 합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	11	48,000	0.02%
Bonded IOB	12	338	3.55%

⑤ 구현(Implementation)·DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999997.000 ns
Hold WHS	0.122 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✔ 비트스트림 생성 완료 — 3,687,022 bytes , 00:31:57 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED[3:0] = value, SW1 = serial_in, N8 = enable로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-04 시프트 레지스터 보드 동작 영상 (evidence/videos/LAB2-04_shift_register.mp4)

[GitHub에서 영상 열기 ↗](#)

5. 실험 15 · PISO

①개요

load면 4비트를 병렬로 저장하고, enable이면 한 칸씩 왼쪽으로 밀며 최상위 비트를 serial_out으로 내 보낸다. load가 enable보다 우선한다.

포트	핀/제어
I/O	clk, rst, load, enable, data_in[3:0] → serial_out, value[3:0]
보드 매핑	LED[7:4] = value, LED0 = serial_out, SW[7:4] = data_in, SW8 = load/shift 선택

②RTL

```
module piso4 (  
    input wire clk, rst, load, enable,  
    input wire [3:0] data_in,  
    output wire serial_out,  
    output reg [3:0] value  
);  
    assign serial_out = value[3];  
    always @(posedge clk) begin  
        if (rst) value <= 4'd0;  
        else if (load) value <= data_in;  
        else if (enable) value <= {value[2:0], 1'b0};  
    end  
endmodule
```

③Behavioral Simulation 결과

 **PASS** VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	114개
종료 시각	976 ns (= 976000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```
LAB2_PASS piso4 checks=114  
C:\Users\UOS\Documents\lab2_05_piso\sim\tb_piso4.sv:19: $finish called at 976000 (1ps)
```

실제 실행 로그 — LAB2_PASS piso4 checks=114 , 976 ns 종료.

④합성(Synthesis) 결과

 **synth_design Complete**

자원	사용량	전체	비율
Slice LUTs	13	48,000	0.03%
Bonded IOB	16	338	4.73%

⑤구현(Implementation)·DRC 결과

✔ write_bitstream Complete

항목	값
Setup WNS	999997.500 ns
Hold WHS	0.121 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✔ 비트스트림 생성 완료 — 3,687,012 bytes , 00:35:33 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED[7:4] = value, LED0 = serial_out, SW[7:4] = data_in, SW8 = load/shift 선택로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-05 PISO 보드 동작 영상 (evidence/videos/LAB2-05_piso.mp4)

[GitHub에서 영상 열기 ↗](#)

6. 실험 16 · Moore 상태 머신

①개요

출력이 상태 그 자체인 Moore 머신. enable·advance가 모두 1인 클럭 에지에서만 00→01→10→00으로 전이한다.

포트	핀/제어
I/O	clk, rst, enable, advance → value[1:0]
보드 매핑	LED7,LED8 = value(상태), SW1 = advance, N8 = enable

②RTL

```

module moore_cycle (
    input wire clk, rst, enable, advance,
    output reg [1:0] value
);
    always @(posedge clk) begin
        if (rst) value <= 2'b00;
        else if (enable && advance) begin
            case (value)
                2'b00: value <= 2'b01;
                2'b01: value <= 2'b10;
                default: value <= 2'b00;
            endcase
        end
    end
endmodule

```

③Behavioral Simulation 결과

 **PASS** VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	23개
종료 시각	226 ns (= 226000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```

LAB2_PASS moore_cycle checks=23
C:\Users\UOS\Documents\lab2_06_moore\sim\tb_moore_cycle.sv:19: $finish called at 226000 (1ps)

```

실제 실행 로그 — LAB2_PASS moore_cycle checks=23 , 226 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	12	48,000	0.03%
Bonded IOB	12	338	3.55%

⑤구현(Implementation)·DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999997.250 ns
Hold WHS	0.122 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✓ 비트스트림 생성 완료 — 3,687,013 bytes , 00:39:13 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED7,LED8 = value(상태), SW1 = advance, N8 = enable로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-06 Moore 상태 머신 보드 동작 영상 (evidence/videos/LAB2-06_moore.mp4)

[GitHub에서 영상 열기 ↗](#)

7. 실험 17 · Mealy 상태 머신

①개요

출력이 상태와 현재 입력에 함께 의존하는 Mealy 머신. state는 클럭 에지에서만 바뀌지만 value는 조합 논리라 입력 변화에 즉시 반응한다.

포트	핀/제어
I/O	clk, rst, enable, bit_in → state, value[1:0]
보드 매핑	LED6 = state, LED7,LED8 = value, SW1 = bit_in, N8 = enable

②RTL

```
module mealy_toggle (  
    input wire clk, rst, enable, bit_in,  
    output reg state,  
    output wire [1:0] value  
);  
    always @(posedge clk) begin  
        if (rst) state <= 1'b0;  
        else if (enable && bit_in) state <= ~state;  
    end  
    assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);  
endmodule
```

③Behavioral Simulation 결과



VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	11개
종료 시각	67 ns (= 67000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```
LAB2_PASS mealy_toggle checks=11  
C:\Users\UOS\Documents\lab2_07_mealy\sim\tb_mealy_toggle.sv:19: $finish called at 67000 (1ps)
```

실제 실행 로그 — LAB2_PASS mealy_toggle checks=11 , 67 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	11	48,000	0.02%
Bonded IOB	12	338	3.55%

⑤구현(Implementation)-DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999997.750 ns
Hold WHS	0.122 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✓ 비트스트림 생성 완료 — 3,687,013 bytes , 00:43:29 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 LED6 = state, LED7,LED8 = value, SW1 = bit_in, N8 = enable로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-07 Mealy 상태 머신 보드 동작 영상 (evidence/videos/LAB2-07_mealy.mp4)

[GitHub에서 영상 열기 ↗](#)

8. 실험 18 · 7세그먼트 자리 스캔

①개요

8자리를 한 번에 한 자리씩 빠르게 켜서(1 ms 점등 + 1 ms 암전) 사람 눈에는 8자리가 동시에 켜진 것처럼 보이게 한다.

포트	핀/제어
I/O	clk, rst, enable, digits[31:0] → select[7:0], segments[7:0], index[2:0]
보드 매핑	8자리 7세그먼트에 SW[7:4]가 첫 자리, 나머지는 1~7 고정 (자동 스캔, 버튼 불필요)

②RTL

```

module segment_scan8 (
    input wire clk, rst, enable,
    input wire [31:0] digits,
    output reg [7:0] select,
    output reg [7:0] segments,
    output reg [2:0] index
);
    reg blank;
    reg [3:0] nibble;
    always @(posedge clk) begin
        if (rst) begin index <= 0; blank <= 1'b1; end
        else if (enable) begin
            blank <= ~blank;
            if (!blank) index <= index + 1'b1;
        end
    end
    always @* begin
        nibble = digits >> (index * 4);
        select = blank ? 8'h00 : (8'h01 << index);
        case (nibble) // LAB1 seg_decoder와 동일한 0~F 7세그 진리표
            4'h0: segments = 8'b11111100; 4'h1: segments = 8'b01100000;
            4'h2: segments = 8'b11011010; 4'h3: segments = 8'b11110010;
            4'h4: segments = 8'b01100110; 4'h5: segments = 8'b10110110;
            4'h6: segments = 8'b10111110; 4'h7: segments = 8'b11100000;
            4'h8: segments = 8'b11111110; 4'h9: segments = 8'b11110110;
            4'ha: segments = 8'b11101110; 4'hb: segments = 8'b00111110;
            4'hc: segments = 8'b10011100; 4'hd: segments = 8'b01111010;
            4'he: segments = 8'b10011110; 4'hf: segments = 8'b10001110;
            default: segments = 8'h00;
        endcase
    end
endmodule

```

③ Behavioral Simulation 결과



VS Code(Icarus)와 Vivado(XSim) 양쪽에서 동일한 결과로 통과했다.

항목	결과
검사 수(cases)	194개
종료 시각	1306 ns (= 1306000 ps)
시뮬레이터	Icarus Verilog 12.0 (VS Code) · Vivado XSim (동일 결과 확인)

```
LAB2_PASS segment_scan8 checks=194
C:\Users\UOS\Documents\lab2_08_segment_scan\sim\tb_segment_scan8.sv:19: $finish called at 1306000 (1ps)
```

실제 실행 로그 — LAB2_PASS segment_scan8 checks=194 , 1306 ns 종료.

④합성(Synthesis) 결과

✓ synth_design Complete

자원	사용량	전체	비율
Slice LUTs	14	48,000	0.03%
Bonded IOB	30	338	8.88%

⑤구현(Implementation)-DRC 결과

✓ write_bitstream Complete

항목	값
Setup WNS	999998.062 ns
Hold WHS	0.119 ns
DRC	오류 0건 · 경고 1건 (CFGBVS-1: Missing CFGBVS and CONFIG_VOLTAGE — 보드 회로도 미확인으로 XDC에 값을 추가하지 않음)

⑥비트스트림 생성 및 보드 검증

✓ 비트스트림 생성 완료 — 3,687,020 bytes , 00:47:28 (2026-09-21)

보드(Combo II-DLD S75, xc7s75fgga484-1)에 프로그램하여 8자리 7세그먼트에 SW[7:4]가 첫 자리, 나머지는 1~7 고정 (자동 스캔, 버튼 불필요)로 정상 동작을 확인했다. 아래 영상은 GitHub 저장소에 커밋된 원본이다.

⑦보드 동작 영상

LAB2-08 7세그먼트 자리 스캔 보드 동작 영상 (evidence/videos/LAB2-08_segment_scan.mp4)

[GitHub에서 영상 열기 ↗](#)